Amazon Linux that includes the Docker packages in its repository, or *Standard Ubuntu Installation*.

Choose an Amazon-provided AMI and launch the *Create Instance Wizard* menu on *AWS Console.* In the *Quick Start* menu, select the Amazon-provided AMI, use the default *t2.micro* instance, configure the *Instance Details* button, select standard choices where default values can be kept, and wait for the Amazon Linux instance to run the SSH instance to install Docker: *ssh -i <path to your private key> ec2-user@<your public IP address>*. Connect to the instance, type *sudo yum install -y docker*; *sudo service docker start* to install and start Docker, and then set up *Security Group* to allow SSH.

Why Docker is considered 'hot' in IT

Docker is widely used by developer teams, systems administrators and QA teams in different environments such as development, testing, pre-production and production, for various reasons. Some of these are:

- Docker can run everywhere regardless of its kernel version.
- It can run on the host or in a container.
- It consists of its own process space, a network interface and can run resources as the root.
- It can share the kernel with the host.
- Docker containers, and their workflow, help developers, sysadmins, QA teams and release engineers to work together to get code into production.
- It is easy to create new containers, enable rapid iteration of applications and increase the visibility of changes.
- Containers are light in weight and quick; they also consist of sub-second launch times, which reduces the time of the development phase, testing and production.
- It can run almost everywhere such as on desktops, virtual machines, physical servers, data centres, and in cloud deployment models such as a private or public cloud.
- It can run on any platform; it can be easily moved from one application environment to another, and vice versa.
- As containers are light in weight, scaling is very easy, i.e., as per needs, they can be launched anytime and can be shut down when not in use.
- Docker containers do not require a hypervisor, so we can get more value out of every server and can potentially reduce the expenditure on equipment and licences.
- As Docker's speed is high, we can make small changes as and when required; moreover, small changes reduce risks and enable more uptime.

Comparison between Docker and virtual machines (VMs)

The significant difference between containers such as Docker and VMs is that the hypervisor abstracts an entire device while containers abstract the operating

Comparing Docker and Virtual Machine

Docker	Virtual Machine
It has a complete operating system installed with the related device drivers	It shares the operating system and device drivers of the host
Small in size; hence, effectively gives faster and better performance	Overhead of memory management and device drivers
Containers abstract the operating system	In a VM, the hypervisor abstracts resources
It runs as an isolated process in user space on the host operating system, and it shares the kernel with other containers	Each application includes not only the application but also the necessary binaries and libraries, and an entire guest operating system
Docker makes it easier to port applications across the cloud using the cloud's VM	The cloud providers use a hypervisor to provide a standard runtime environment for VMs

Comparing Docker and Vagrant

Docker	Vagrant
It is a virtual environment tool	It is a virtual management tool
Docker was open sourced in 2013	The open source version was released in 2010
It can configure management systems to provision the containers	It can coordinate with a configuration management software
It provides ways to define operating system installations, but greatly differs in the technology used for this purpose	Vagrant can coordinate with configuration management software to continue the process of installation when the operating system's installer has finished its role